JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractQueuedLongSynchronizer

java.lang.Object
  java.util.concurrent.locks.AbstractOwnableSynchronizer
    java.util.concurrent.locks.AbstractQueuedLongSynchronizer

All Implemented Interfaces:

Serializable

```
public abstract class AbstractQueuedLongSynchronizer
extends AbstractOwnableSynchronizer
implements Serializable
```

A version of `AbstractQueuedSynchronizer` in which synchronization state is maintained as a `long`. This class has exactly the same structure, properties, and methods as `AbstractQueuedSynchronizer` with the exception that all state-related parameters and results are defined as `long` rather than `int`. This class may be useful when creating synchronizers such as multilevel locks and barriers that require 64 bits of state.

See `AbstractQueuedSynchronizer` for usage notes and examples.

Since:

1.6

See Also:

Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
class	**AbstractQueuedLongSynchr**	Condition implementation for a `AbstractQueuedLongSynchronizer` serving as the basis of a `Lock` implementation.

Constructor Summary

Constructors

Constructor	Description
AbstractQueuedLongSynchronizer()	Constructor for subclasses to call.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
final void	acquire(long arg)	Acquires in exclusive mode, ignoring interrupts.
final void	acquireInterruptibly (long arg)	Acquires in exclusive mode, aborting if interrupted.
final void	acquireShared(long arg)	Acquires in shared mode, ignoring interrupts.
final void	acquireSharedInterruptib (long arg)	Acquires in shared mode, aborting if interrupted.
protected final boolean	compareAndSetState (long expect, long update)	Atomically sets synchronization state to the given updated value if the current state value equals the expected value.
final Collection <Thread>	getExclusiveQueuedThread	Returns a collection containing threads that may be waiting to acquire in exclusive mode.
final Thread	getFirstQueuedThread()	Returns the first (longest-waiting) thread in the queue, or null if no threads are currently queued.
final Collection <Thread>	getQueuedThreads()	Returns a collection containing threads that may be waiting to acquire.
final int	getQueueLength()	

		Returns an estimate of the number of threads waiting to acquire.
final Collection<Thread>	**getSharedQueuedThreads**()	Returns a collection containing threads that may be waiting to acquire in shared mode.
protected final long	**getState**()	Returns the current value of synchronization state.
final Collection<Thread>	**getWaitingThreads**(AbstractQueuedLongSynch	Returns a collection containing those threads that may be waiting on the given condition associated with this synchronizer.
final int	**getWaitQueueLength**(AbstractQueuedLongSynch	Returns an estimate of the number of threads waiting on the given condition associated with this synchronizer.
final boolean	**hasContended**()	Queries whether any threads have ever contended to acquire this synchronizer; that is, if an acquire method has ever blocked.
final boolean	**hasQueuedPredecessors**()	Queries whether any threads have been waiting to acquire longer than the current thread.
final boolean	**hasQueuedThreads**()	Queries whether any threads are waiting to acquire.
final boolean	**hasWaiters**(AbstractQueuedLongSynch	Queries whether any threads are waiting on the given condition

		associated with this synchronizer.
protected boolean	**isHeldExclusively**()	Returns true if synchronization is held exclusively with respect to the current (calling) thread.
final boolean	**isQueued**(Thread thread)	Returns true if the given thread is currently queued.
final boolean	**owns**(AbstractQueuedLongSynch	Queries whether the given ConditionObject uses this synchronizer as its lock.
final boolean	**release**(long arg)	Releases in exclusive mode.
final boolean	**releaseShared**(long arg)	Releases in shared mode.
protected final void	**setState**(long newState)	Sets the value of synchronization state.
String	**toString**()	Returns a string identifying this synchronizer, as well as its state.
protected boolean	**tryAcquire**(long arg)	Attempts to acquire in exclusive mode.
final boolean	**tryAcquireNanos**(long arg, long nanosTimeout)	Attempts to acquire in exclusive mode, aborting if interrupted, and failing if the given timeout elapses.
protected long	**tryAcquireShared**(long arg)	Attempts to acquire in shared mode.

final boolean	tryAcquireSharedNanos(long arg, long nanosTimeout)	Attempts to acquire in shared mode, aborting if interrupted, and failing if the given timeout elapses.
protected boolean	tryRelease(long arg)	Attempts to set the state to reflect a release in exclusive mode.
protected boolean	tryReleaseShared(long arg)	Attempts to set the state to reflect a release in shared mode.

Methods declared in class AbstractOwnableSynchronizer

getExclusiveOwnerThread, setExclusiveOwnerThread

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

AbstractQueuedLongSynchronizer

```
public AbstractQueuedLongSynchronizer()
```

Constructor for subclasses to call.

Method Details

getState

```
protected final long getState()
```

Returns the current value of synchronization state. This operation has memory semantics of a `volatile` read.

Returns:

current state value

setState

```
protected final void setState(long newState)
```

Sets the value of synchronization state. This operation has memory semantics of a `volatile` write.

Parameters:

`newState` - the new state value

compareAndSetState

```
protected final boolean compareAndSetState(long expect,
                                           long update)
```

Atomically sets synchronization state to the given updated value if the current state value equals the expected value. This operation has memory semantics of a `volatile` read and write.

Parameters:

`expect` - the expected value

`update` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

tryAcquire

```
protected boolean tryAcquire(long arg)
```

Attempts to acquire in exclusive mode. This method should query if the state of the object permits it to be acquired in the exclusive mode, and if so to acquire it.

This method is always invoked by the thread performing acquire. If this method reports failure, the acquire method may queue the thread, if it is not already queued, until it is signalled by a release from some other thread. This can be used to implement method `Lock.tryLock()`.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the acquire argument. This value is always the one passed to an acquire method, or is the value saved on entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if successful. Upon success, this object has been acquired.

Throws:

`IllegalMonitorStateException` - if acquiring would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if exclusive mode is not supported

tryRelease

```
protected boolean tryRelease(long arg)
```

Attempts to set the state to reflect a release in exclusive mode.

This method is always invoked by the thread performing release.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the release argument. This value is always the one passed to a release method, or the current state value upon entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if this object is now in a fully released state, so that any waiting threads may attempt to acquire; and `false` otherwise.

Throws:

`IllegalMonitorStateException` - if releasing would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if exclusive mode is not supported

tryAcquireShared

```
protected long tryAcquireShared(long arg)
```

Attempts to acquire in shared mode. This method should query if the state of the
object permits it to be acquired in the shared mode, and if so to acquire it.

This method is always invoked by the thread performing acquire. If this method reports failure, the acquire method may queue the thread, if it is not already queued, until it is signalled by a release from some other thread.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the acquire argument. This value is always the one passed to an acquire method, or is the value saved on entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

a negative value on failure; zero if acquisition in shared mode succeeded but no subsequent shared-mode acquire can succeed; and a positive value if acquisition in shared mode succeeded and subsequent shared-mode acquires might also succeed, in which case a subsequent waiting thread must check availability. (Support for three different return values enables this method to be used in contexts where acquires only sometimes act exclusively.) Upon success, this object has been acquired.

Throws:

`IllegalMonitorStateException` - if acquiring would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if shared mode is not supported

tryReleaseShared

```
protected boolean tryReleaseShared(long arg)
```

Attempts to set the state to reflect a release in shared mode.

This method is always invoked by the thread performing release.

The default implementation throws `UnsupportedOperationException`.

Parameters:

`arg` - the release argument. This value is always the one passed to a release method, or the current state value upon entry to a condition wait. The value is otherwise uninterpreted and can represent anything you like.

Returns:

`true` if this release of shared mode may permit a waiting acquire (shared or exclusive) to succeed; and `false` otherwise

Throws:

`IllegalMonitorStateException` - if releasing would place this synchronizer in an illegal state. This exception must be thrown in a consistent fashion for synchronization to work correctly.

`UnsupportedOperationException` - if shared mode is not supported

isHeldExclusively

```
protected boolean isHeldExclusively()
```

Returns `true` if synchronization is held exclusively with respect to the current (calling) thread. This method is invoked upon each call to a `AbstractQueuedLongSynchronizer.ConditionObject` method.

The default implementation throws `UnsupportedOperationException`. This method is invoked internally only within `AbstractQueuedLongSynchronizer.ConditionObject` methods, so need not be defined if conditions are not used.

Returns:

`true` if synchronization is held exclusively; `false` otherwise

Throws:

`UnsupportedOperationException` - if conditions are not supported

acquire

```
public final void acquire(long arg)
```

Acquires in exclusive mode, ignoring interrupts. Implemented by invoking at least once `tryAcquire(long)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(long)` until success. This method can be used to implement method `Lock.lock()`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(long)` but is otherwise uninterpreted and can represent anything you like.

acquireInterruptibly

```
public final void acquireInterruptibly(long arg)
                                throws InterruptedException
```

Acquires in exclusive mode, aborting if interrupted. Implemented by first checking interrupted status, then invoking at least once `tryAcquire(long)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(long)` until success or the thread is interrupted. This method can be used to implement method `Lock.lockInterruptibly()`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(long)` but is otherwise uninterpreted and can represent anything you like.

Throws:

`InterruptedException` - if the current thread is interrupted

tryAcquireNanos

```
public final boolean tryAcquireNanos(long arg,
                                     long nanosTimeout)
                              throws InterruptedException
```

Attempts to acquire in exclusive mode, aborting if interrupted, and failing if the given timeout elapses. Implemented by first checking interrupted status, then invoking at least once `tryAcquire(long)`, returning on success. Otherwise, the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquire(long)` until success or the thread is interrupted or the timeout elapses. This method can be used to implement method `Lock.tryLock(long, TimeUnit)`.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquire(long)` but is otherwise uninterpreted and can represent anything you like.

`nanosTimeout` - the maximum number of nanoseconds to wait

Returns:

`true` if acquired; `false` if timed out

Throws:

`InterruptedException` - if the current thread is interrupted

release

```
public final boolean release(long arg)
```

Releases in exclusive mode. Implemented by unblocking one or more threads if `tryRelease(long)` returns true. This method can be used to implement method `Lock.unlock()`.

Parameters:

`arg` - the release argument. This value is conveyed to `tryRelease(long)` but is otherwise uninterpreted and can represent anything you like.

Returns:

the value returned from `tryRelease(long)`

acquireShared

```
public final void acquireShared(long arg)
```

Acquires in shared mode, ignoring interrupts. Implemented by first invoking at least once `tryAcquireShared(long)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(long)` until success.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(long)` but is otherwise uninterpreted and can represent anything you like.

acquireSharedInterruptibly

```
public final void acquireSharedInterruptibly(long arg)
                                       throws InterruptedException
```

Acquires in shared mode, aborting if interrupted. Implemented by first checking interrupted status, then invoking at least once `tryAcquireShared(long)`, returning on success. Otherwise the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(long)` until success or the thread is interrupted.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(long)` but is otherwise uninterpreted and can represent anything you like.

Throws:

`InterruptedException` - if the current thread is interrupted

tryAcquireSharedNanos

```
public final boolean tryAcquireSharedNanos(long arg,
                                           long nanosTimeout)
                                    throws InterruptedException
```

Attempts to acquire in shared mode, aborting if interrupted, and failing if the given timeout elapses. Implemented by first checking interrupted status, then invoking at least once `tryAcquireShared(long)`, returning on success. Otherwise, the thread is queued, possibly repeatedly blocking and unblocking, invoking `tryAcquireShared(long)` until success or the thread is interrupted or the timeout elapses.

Parameters:

`arg` - the acquire argument. This value is conveyed to `tryAcquireShared(long)` but is otherwise uninterpreted and can represent anything you like.

`nanosTimeout` - the maximum number of nanoseconds to wait

Returns:

`true` if acquired; `false` if timed out

Throws:

`InterruptedException` - if the current thread is interrupted

releaseShared

```
public final boolean releaseShared(long arg)
```

Releases in shared mode. Implemented by unblocking one or more threads if `tryReleaseShared(long)` returns true.

Parameters:

`arg` - the release argument. This value is conveyed to `tryReleaseShared(long)` but is otherwise uninterpreted and can represent anything you like.

Returns:

the value returned from `tryReleaseShared(long)`

hasQueuedThreads

```
public final boolean hasQueuedThreads()
```

Queries whether any threads are waiting to acquire. Note that because cancellations due to interrupts and timeouts may occur at any time, a `true` return does not guarantee that any other thread will ever acquire.

Returns:

`true` if there may be other threads waiting to acquire

hasContended

```
public final boolean hasContended()
```

Queries whether any threads have ever contended to acquire this synchronizer; that is, if an acquire method has ever blocked.

In this implementation, this operation returns in constant time.

Returns:

`true` if there has ever been contention

getFirstQueuedThread

```
public final Thread getFirstQueuedThread()
```

Returns the first (longest-waiting) thread in the queue, or `null` if no threads are currently queued.

In this implementation, this operation normally returns in constant time, but may iterate upon contention if other threads are concurrently modifying the queue.

Returns:

the first (longest-waiting) thread in the queue, or `null` if no threads are currently queued

isQueued

`public final boolean isQueued(Thread thread)`

Returns true if the given thread is currently queued.

This implementation traverses the queue to determine presence of the given thread.

Parameters:

`thread` - the thread

Returns:

`true` if the given thread is on the queue

Throws:

`NullPointerException` - if the thread is null

hasQueuedPredecessors

`public final boolean hasQueuedPredecessors()`

Queries whether any threads have been waiting to acquire longer than the current thread.

An
invocation of this method is equivalent to (but may be more efficient than):

```
getFirstQueuedThread() != Thread.currentThread()
  && hasQueuedThreads()
```

Note that because cancellations due to interrupts and timeouts may occur at any time, a `true` return does not guarantee that some other thread will acquire before the current thread. Likewise, it is possible for another thread to win a race to enqueue after this method has returned `false`, due to the queue being empty.

This method is designed to be used by a fair synchronizer to avoid barging. Such a synchronizer's `tryAcquire(long)` method should return `false`, and its `tryAcquireShared(long)` method should return a negative value, if this method returns `true` (unless this is a reentrant acquire). For example, the `tryAcquire` method for a fair, reentrant, exclusive mode synchronizer might look like this:

```
protected boolean tryAcquire(long arg) {
  if (isHeldExclusively()) {
    // A reentrant acquire; increment hold count
```

```
    return true;
  } else if (hasQueuedPredecessors()) {
    return false;
  } else {
    // try to acquire normally
  }
}
```

Returns:

`true` if there is a queued thread preceding the current thread, and `false` if the current thread is at the head of the queue or the queue is empty

Since:

1.7

getQueueLength

```
public final int getQueueLength()
```

Returns an estimate of the number of threads waiting to acquire. The value is only an estimate because the number of threads may change dynamically while this method traverses internal data structures. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

the estimated number of threads waiting to acquire

getQueuedThreads

```
public final Collection<Thread> getQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive monitoring facilities.

Returns:

the collection of threads

getExclusiveQueuedThreads

```
public final Collection<Thread> getExclusiveQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire in exclusive mode. This has the same properties as `getQueuedThreads()` except that it only returns those threads waiting due to an exclusive acquire.

Returns:

the collection of threads

getSharedQueuedThreads

```
public final Collection<Thread> getSharedQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire in shared mode. This has the same properties as `getQueuedThreads()` except that it only returns those threads waiting due to a shared acquire.

Returns:

the collection of threads

toString

```
public String toString()
```

Returns a string identifying this synchronizer, as well as its state. The state, in brackets, includes the String `"State ="` followed by the current value of `getState()`, and either `"nonempty"` or `"empty"` depending on whether the queue is empty.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this synchronizer, as well as its state

owns

```
public final boolean owns
(AbstractQueuedLongSynchronizer.ConditionObject condition)
```

Queries whether the given ConditionObject uses this synchronizer as its lock.

Parameters:

`condition` - the condition

Returns:

`true` if owned

Throws:

`NullPointerException` - if the condition is null

hasWaiters

```
public final boolean hasWaiters
(AbstractQueuedLongSynchronizer.ConditionObject condition)
```

Queries whether any threads are waiting on the given condition associated with this synchronizer. Note that because timeouts and interrupts may occur at any time, a `true` return does not guarantee that a future `signal` will awaken any threads. This method is designed primarily for use in monitoring of the system state.

Parameters:

`condition` - the condition

Returns:

`true` if there are any waiting threads

Throws:

`IllegalMonitorStateException` - if exclusive synchronization is not held

`IllegalArgumentException` - if the given condition is not associated with this synchronizer

`NullPointerException` - if the condition is null

getWaitQueueLength

```
public final int getWaitQueueLength
(AbstractQueuedLongSynchronizer.ConditionObject condition)
```

Returns an estimate of the number of threads waiting on the given condition associated with this synchronizer. Note that because timeouts and interrupts may occur at any time, the estimate serves only as an upper bound on the actual number of waiters. This method is designed for use in monitoring system state, not for synchronization control.

Parameters:

`condition` - the condition

Returns:

the estimated number of waiting threads

Throws:

`IllegalMonitorStateException` - if exclusive synchronization is not held

`IllegalArgumentException` - if the given condition is not associated with this synchronizer

`NullPointerException` - if the condition is null

getWaitingThreads

```
public final Collection<Thread> getWaitingThreads
(AbstractQueuedLongSynchronizer.ConditionObject condition)
```

Returns a collection containing those threads that may be waiting on the given condition associated with this synchronizer. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order.

Parameters:

`condition` - the condition

Returns:

the collection of threads

Throws:

`IllegalMonitorStateException` - if exclusive synchronization is not held

`IllegalArgumentException` - if the given condition is not associated with this synchronizer

`NullPointerException` - if the condition is null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.